

name = rahid

id = 15336

paper = computer vision mid lab paper

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

path = r"C:\Users\syedi\Downloads\rahid\download.jfif"

if not os.path.exists(path):
    print("File not found. Check name or extension.")
else:
    img = cv2.imread(path)

    if img is None:
        print("Image loading failed.")
    else:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        print("Original Shape:", img.shape)
        print("Grayscale Shape:", gray.shape)

        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.figure(figsize=(10,5))

        plt.subplot(1,2,1)
        plt.title("Original Image")
        plt.imshow(img_rgb)
        plt.axis('off')

        plt.subplot(1,2,2)
        plt.title("Grayscale Image")
        plt.imshow(gray, cmap='gray')
        plt.axis('off')

    plt.show()
```

Original Shape: (148, 221, 3)

Grayscale Shape: (148, 221)

Original Image



Grayscale Image



Question 2

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

cap = cv2.VideoCapture(0)

ret, frame = cap.read()
cap.release()

if not ret:
    print("Camera not working")
else:
    bright = cv2.convertScaleAbs(frame, alpha=1, beta=60)

    contrast = cv2.convertScaleAbs(bright, alpha=1.6, beta=0)

    original = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    enhanced = cv2.cvtColor(contrast, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10,5))

    plt.subplot(1,2,1)
    plt.title("Original Capture")
    plt.imshow(original)
    plt.axis('off')

    plt.subplot(1,2,2)
    plt.title("Enhanced Image")
    plt.imshow(enhanced)
    plt.axis('off')

    plt.show()
```

Original Capture



Enhanced Image



question 3

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

path1 = r"C:\Users\syedi\Downloads\rahid\3.jpg"
path2 = r"C:\Users\syedi\Downloads\rahid\3a.jpg"

img1 = cv2.imread(path1)
img2 = cv2.imread(path2)

if img1 is None or img2 is None:
    print("Error loading images")
else:
    img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

    blended = cv2.addWeighted(img1, 0.5, img2, 0.5, 0)

    blended[0:50, 0:50] = [0, 0, 255]

    cv2.rectangle(blended, (0,0), (50,50), (0,255,0), 2)

    img_rgb = cv2.cvtColor(blended, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(6,5))
    plt.title("Blended + Pixel Modified Image")
    plt.imshow(img_rgb)
    plt.axis('off')
    plt.show()
```

Blended + Pixel Modified Image



question 4

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

path = r"C:\Users\syedi\Downloads\rahid\4.jfif"
img = cv2.imread(path)

if img is None:
    print("Error: image not found")
else:
    # Filters
    avg = cv2.blur(img, (5,5))                # Averaging
    gauss = cv2.GaussianBlur(img, (5,5), 0)    # Gaussian
    median = cv2.medianBlur(img, 5)            # Median

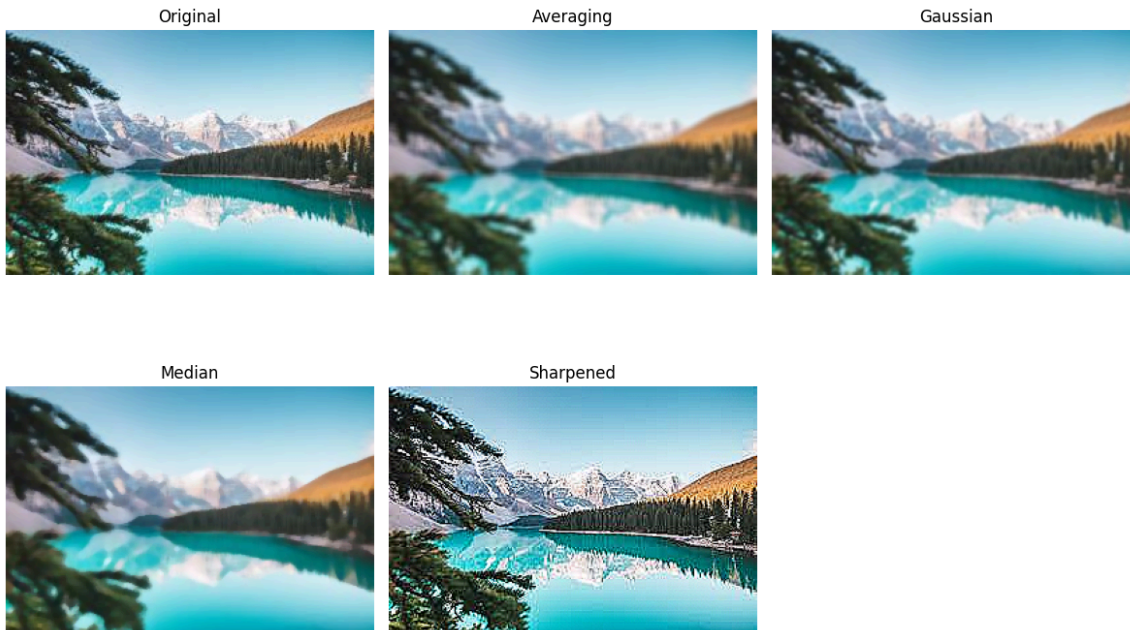
    # Sharpening kernel
    kernel = np.array([[0,-1,0],
                       [-1,5,-1],
                       [0,-1,0]])
    sharp = cv2.filter2D(img, -1, kernel)

    # Convert to RGB for display
    imgs = [img, avg, gauss, median, sharp]
    titles = ["Original", "Averaging", "Gaussian", "Median", "Sharpened"]

    plt.figure(figsize=(12,8))

    for i in range(5):
        plt.subplot(2,3,i+1)
        plt.title(titles[i])
        plt.imshow(cv2.cvtColor(imgs[i], cv2.COLOR_BGR2RGB))
        plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



```
import cv2
import numpy as np
import matplotlib.pyplot as plt

path = r"C:\Users\syedi\Downloads\rahid\5.jfif"
img = cv2.imread(path)

if img is None:
    print("Image not found")
else:
    original = img.copy()

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    _, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

    edges = cv2.Canny(gray, 50, 150)

    lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=100,
                             minLineLength=50, maxLineGap=10)

    if lines is not None:
        for line in lines:
            x1,y1,x2,y2 = line[0]
            cv2.line(img, (x1,y1), (x2,y2), (0,255,0), 2)

    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, 1, 30,
                                param1=50, param2=30,
                                minRadius=10, maxRadius=100)

    if circles is not None:
```

```

circles = np.uint16(np.around(circles))
for c in circles[0,:]:
    cv2.circle(img, (c[0], c[1]), c[2], (255,0,0), 2)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1,2,2)
plt.title("Segmented + Detected Shapes")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.show()

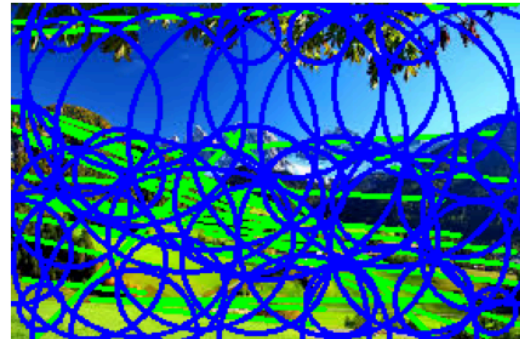
plt.figure(figsize=(6,4))
plt.title("Threshold Segmentation")
plt.imshow(thresh, cmap='gray')
plt.axis('off')
plt.show()

```

Original Image



Segmented + Detected Shapes



Threshold Segmentation

